

MENTOR-RL Methods

Peter Kruse

June 2026

0.1 Introduction

Mechanistic interpretation identifies relationships among molecular entities such as genes, proteins, and RNA. Systems biology studies their interactions to find *modules*, or groups of functionally related entities, and characterize the mechanisms underlying their behavior [17, 5]. Because these interactions span scales and regulatory layers, modern analysis uses multiplex networks with the same genes connected by different edge types [4, 30, 14] and propagation methods such as random walk with restart (RWR) to identify functional neighborhoods [23, 13]. Such analyses support disease-mechanism discovery, drug-target prioritization, and polygenic-phenotype interpretation [28]. These methods build on one another: iterative random forest constructs predictive networks from data [6, 12], diffusion identifies functional neighborhoods within networks, and MENTOR [46] organizes the resulting distances into a module hierarchy. Despite providing a structured view of the interactome, these methods still produce modules and candidate lists that are too large to explore systematically.

Large language models (LLMs) can synthesize large amounts of information, and several techniques make them suitable for mechanistic exploration. Intermediate reasoning, such as chain-of-thought and tree-of-thoughts, improves reliability [51, 24], while external tool use provides deterministic verification and provenance [41, 53]. Applying explicit self-verification to these steps has been shown to further improve factuality [27, 44]. Applied to biology, such techniques have equipped models with gene-function embeddings [10], database-supported self-verifying pipelines [50], and multi-agent systems for mechanistic synthesis [49, 26]. However, these systems reach that behavior by using frontier-model inference, so their cost grows with exploration length and reasoning depth. Furthermore, none are trained end-to-end as a policy over verifiable rewards.

Reinforcement learning supplies the machinery to close the gap between frontier and open-weight models. Direct preference optimization (DPO) aligns a policy with pairwise judgments [11, 35, 38], while policy-gradient methods such as proximal policy optimization and group relative policy optimization (GRPO) optimize scalar rewards over complete rollouts [42, 43]. Verifiable rewards can elicit long-horizon reasoning [16], and related methods train tool-using agents for browsing, APIs, and multistep web tasks [32, 37, 36]. Although these methods have shown promise for a variety of use cases, they have not yet been applied to a policy grounded in network-derived biological targets.

Current systems also treat networks as static objects to query and encode rather than learning the rules they obey, limiting generalization to new cell types, tissues, or diseases. Sequence models trained on synthetic tasks can instead form internal, causal models of a generating process and generalize to unseen states [25]. These compact, recurring rules can be parameterized by model weights, even though the large multiplex structures themselves are too large. Thus, it is possible for a language model to learn network structure while keeping biological edges, walk scores, and

distances in a deterministic runtime. Learning such a world model also requires dense sampling of the generation process, since fixed templates over a single network can be satisfied through memorization.

We therefore propose MENTOR-RL, an open-weight model trained on a structured lattice of multiplex networks. In every context, relevant predictive expression network (PEN) layers define the gene universe, the remaining layers are induced on that universe, and all layers are stacked. Varying the cell type, tissue, or brain region changes the network while holding this construction rule fixed. This *multiplex lattice* supplies the dense sampling that a single network cannot. Across contexts, the operator that maps layer composition to structure stays fixed. We expect hierarchies to differ across contexts; this divergence is biological signal, and recovering it tests whether the model learned the operator.

However, a hierarchy alone cannot serve as the world model, because it is a lossy abstraction of the multiplex. Two clades far apart in a tree may still be joined by short, biologically supported paths through bridge nodes, which is why a signal such as drug-target enrichment can appear in distant branches but act through a single connected system. We therefore train the model on network structure itself, so it can reason about the cross-clade connectivity that hierarchy suppresses. The hierarchy is used to prioritize candidates, while the network is used to confirm them.

This proposal pursues four objectives. First, we define an ontology-grounded multiplex lattice, serving as a training substrate for supervised fine-tuning (SFT). Next, we propose to train a world model on three structural views using typed, runtime-validated targets. After world model training, we plan to post-train an exploration agent via reinforcement learning with deterministic rewards. Finally, we will test the world model claim through two flagships: hierarchy generation for an unseen context, and projection of a seed neighborhood onto the hierarchy followed by network-connectivity recovery. We employ activation probes to test whether the recovered structure is causal. Together these objectives aim to make mechanistic discovery broader, faster, and more context-aware.

0.2 Proposed Methods

0.2.1 Multiplex Lattice

In this section, we define the multiplex lattice that serves as the substrate for our mechanistic world model. The lattice comprises multiplex networks in which entities such as genes and proteins are nodes and their context-specific interactions are edges, with each interaction type being represented in a different layer. For each context, we take every available PEN layer and define its *gene universe* as the union of genes contained in those layers. The remaining non-PEN layers are then *induced* on that universe, restricting them to genes present in the PEN layers and the edges among them. Finally, all layers in the context are *stacked* to form a multiplex network

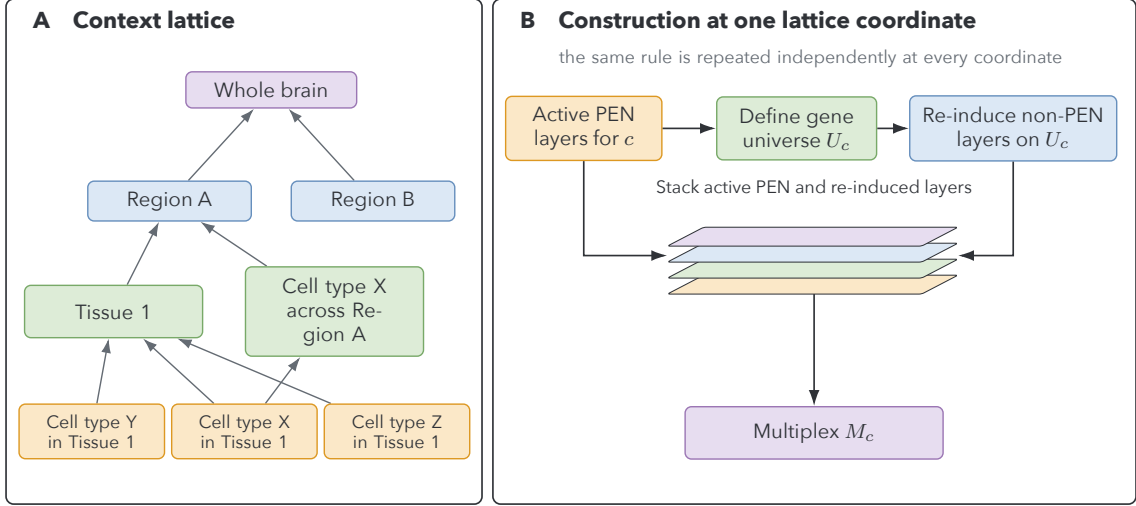


Figure 1: The multiplex lattice and its shared construction rule. Panel A gives a generic partial-order example. A cell-type-within-tissue context belongs to both its tissue context and its cell-type-across-region context. Panel B shows the construction at one coordinate. Active PEN layers define the gene universe, non-PEN layers are re-induced on that universe, and both layer sets are stacked into a distinct multiplex.

that captures multilayer interactions among genes. Thus, there is no single shared backbone across the lattice; even layers drawn from a common source become context-specific once induced. Two sampled points in the lattice may contain different nodes, edges, and therefore hierarchies. For the scope of this proposal, we build the lattice on the brain, for which we hold a pre-existing whole-brain multiplex assembled by the same rule. The brain multiplex combines PEN layers over the brain gene universe with a HumanNet V3 backbone [22] induced on it.

The lattice comprises a tiered structure that captures the relationships between different contexts. We visualize an example brain multiplex lattice in Figure 1. Its most granular component is a single cell type, such as an excitatory neuron, inhibitory neuron, or glial cell, within one tissue of one region. Combining cell types yields a tissue-level multiplex; combining tissues yields a region-level multiplex such as the prefrontal cortex, hippocampus, or cerebellum; and combining regions builds up to the whole-brain multiplex described above. Every tier is therefore a multiplex in its own right, built by the same rule. These tiers do not form a strict tree, since a context can belong to more than one grouping at the tier above it. For example, an astrocyte in CA1 sits beneath both the all-cell-types node for CA1 and the astrocyte node for the hippocampus as a whole. Section 0.2.2 formalizes the resulting structure. Disease networks for anxiety, suicide, and pan-substance-use-disorder are points in this same lattice, which makes transfer meaningful. A model that has learned the construction operator can be applied to diseases directly.

Higher tiers are built by applying the same rule to a union of contexts. We take the union of the parts’ gene universes, re-induce the non-PEN layers on it, and stack the parts’ PEN layers on top. This process makes lattice sampling highly combinatorial, as changing a single PEN layer changes two things at once. The gene universe grows or shrinks, and the backbone is re-induced to match. A pair of multiplexes that differ by one layer therefore also differ by the backbone that the layer induces. Measuring the effect of a layer on its own requires holding the gene universe fixed, which we will test via controlled ablation. The lattice also holds far more combinations than we can build, so we note that tier counts and sampling rates are provisional until divergence validation shows how much contexts actually differ.

0.2.2 Context Ontology

To train MENTOR-RL reproducibly, we define a *context ontology* that specifies relationships among contexts. This ontology is supplied, versioned metadata that lets us name, version, and align the lattice with external sources. We use several grounded ontologies to define the context hierarchy. Anatomy is defined by UBERON [31], while cell types are defined by the Cell Ontology [15] and extended by the Provisional Cell Ontology [47] for transcriptomic brain types not yet represented in the Cell Ontology. Developmental stages are defined by HsapDv [7], anatomy-to-cell-type mappings by the Human Reference Atlas [9], and node metadata by the CELLxGENE schema [1]. Because the Human Reference Atlas covers the whole body, this paradigm can later extend beyond the brain.

Contexts form a partial order instead of a tree because some cell types appear in multiple tissues and some tissues appear in multiple regions. We therefore define a *join* operation, where the join of two contexts is the smallest context containing both. Joining two contexts produces the union multiplex described in Section 0.2.1, which links the ontology to the construction rule.

We supply the ontology as input and do not store it in the model weights through training. Instead, the model is trained to navigate the ontology by returning a context’s parents, children, siblings, joins, and distances. Because only terms change while the operators remain static across the lattice, we expect the model to generalize to unseen contexts.

0.2.3 Three Structural Views

To represent this ontology-based lattice structure as a training target, we define three equally weighted structural views of a multiplex network: bottom-up random-walk-with-restart lines-of-evidence (RWR-LOE) neighborhoods, top-down MENTOR-EV-derived hierarchies, and cross-module network connectivity. Each view represents the same underlying multiplex with different biases.

The RWR-LOE view is a bottom-up representation of a multiplex that captures the local topology around each node [21]. For each node in a given multiplex, we

perform a random walk with restart over all layers of the multiplex, treating each layer as an independent line of evidence. The walk returns a probability distribution over all nodes in the network, with higher probabilities representing closer proximity to the seed. We then threshold the distribution using a geometric elbow, which cuts the score-versus-rank curve at its bend. The genes above the cutoff form a module of entities that are proximal to the seed over multiple lines of evidence. From RWR-LOE modules, we can learn several operator types, including rank and score lookup, rank comparison, and relations between modules.

MENTOR-EV-derived modules represent an alternative, top-down view of the multiplex. MENTOR-EV applies MENTOR [46] using every gene in the multiplex as a seed, so the resulting hierarchy spans the full gene universe, not a supplied gene set. To create MENTOR-EV modules, we first use RWR to compute the probability distribution for each node in the multiplex. We then compute pairwise similarity between RWR vectors using Spearman correlation and subtract those correlations from one to construct a dissimilarity matrix over all nodes. Finally, hierarchical clustering produces a dendrogram in which leaves are genes and each internal node defines a *clade*, the set of genes beneath it. Cutting the dendrogram at a given merge height produces a family of nested modules and controls their granularity. From MENTOR-EV modules, we can learn several operator types, including module membership, module comparison, lowest common ancestors, and distance between modules.

Every one of these targets is read from a single dendrogram. The structural targets are alignment-free and do not require two hierarchies to agree, although we compare derived divergence summaries across contexts. This is what makes the divergence described in Section 0.2.1 trainable instead of an obstacle. Hierarchies differ across contexts, but each one supplies its own targets.

However, RWR-LOE and MENTOR-EV modules are both lossy abstractions of the underlying multiplex structure. Two clades on distant branches of the dendrogram may still be connected by short paths through a small number of bridge nodes, and a neighborhood cut at the elbow drops genes that remain reachable in a few hops. As such, we also include learnable network connectivity operators as a third view, which include cross-clade path recovery, connector and bridge node identification, path decomposition by supporting layer, and cohesion tested against a null. These operators allow the model to reason about the underlying multiplex structure and recover connectivity that is not captured by the other two views.

0.2.4 Stage 1: Multiplex World Model

In Stage 1, we train a world model to internalize the multiplex lattice by learning the operators that generate its structure. The model learns using a deterministic runtime that answers graph queries, so numeric results come from the runtime, while the model learns to reason about the operators that produce them. A ten-step curriculum introduces the operators in order, and every training example is

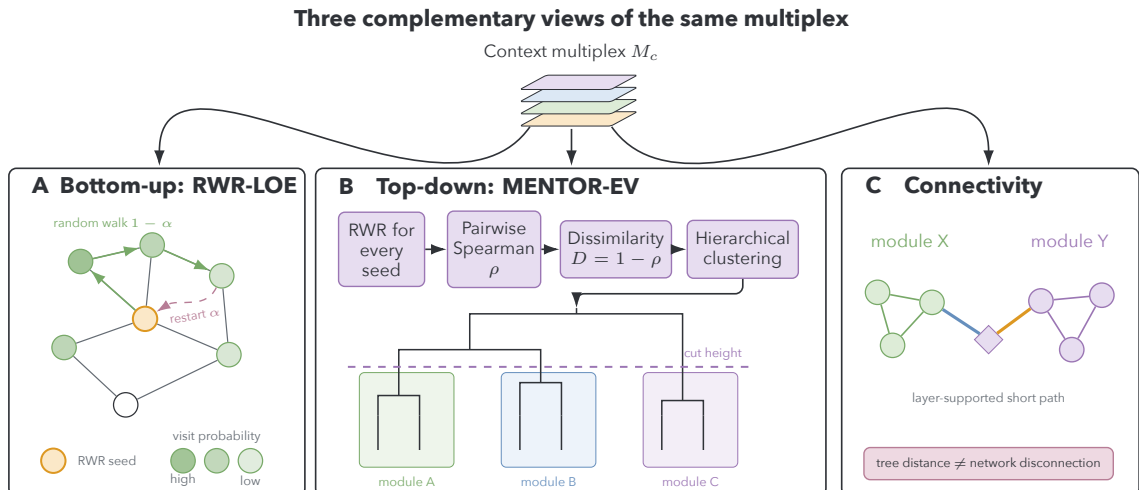


Figure 2: Three structural views derived from the same context multiplex. RWR-LOE supplies bottom-up neighborhoods, MENTOR-EV supplies top-down nested clades, and direct network operators recover paths and bridge nodes suppressed by either abstraction. The views are complementary targets, not interchangeable descriptions.

generated and validated by the runtime. Stage 1 will produce the checkpoint that initializes agent training in Section 0.2.5.

Deterministic Runtime and Tool Contract

To make the lattice environment compatible with the training pipeline, we use a compiled native multiplex library exposed to Python through `ctypes` bindings [20]. Corpus construction, training, and optimization code are implemented in Python [40], while graph storage and graph operations are handled by the runtime for efficiency on large multiplexes. Multiplex networks exist as raw binaries described by JSON manifests; this format is confined to native storage and execution. Model-facing tools instead accept structured biological requests and return compact, structured observations from the queried multiplex. The manifest carries the layer names, node metadata, and build metadata needed to load the graph.

The runtime uses a set of optimized multiplex functions developed as part of the MENTOR [46] and MENTOR-IA [49] projects, known as RWR++. Core operators include neighborhood retrieval, shortest-path search, random walk with restart on both monoplex and multiplex views, and subgraph induction. These operators provide the primary graph-facing actions available to the model.

We structure graph tools as learnable, interpretable operators according to three rules. First, each tool accepts only biologically meaningful arguments, such as gene identifiers, layer names, and module identifiers. Thus, when learning to produce a tool call, the model reasons about the biology instead of the file system or command-line

interface. Second, we defer numeric computation to the runtime; because tokenization makes precise floating-point values difficult for the model to produce, the model calls a tool to compute any required statistic. Third, we enforce determinism, so the same arguments, graph version, layer scope, and parameter version always return identical output. This enables tool-call caching and reproducible scoring.

Runtime operators expose compact graph summaries and provenance metadata to the model instead of raw vectors or matrices. These observations record the relevant layer identity, operator parameters, and source information needed to trace the final interpretation back to the graph evidence used to construct it. To ensure consistency across serialized multiplex files, C++ runtime objects, Python wrappers, and logged training artifacts, all nodes are represented in a single canonical identifier space and all edges are referenced using deterministic, layer-aware identifiers. Table 1 lists the model-facing tool vocabulary used in the curriculum.

Supervised Curriculum

To train the model to internalize the multiplex lattice, we design a supervised curriculum that teaches it to navigate the lattice and learn the operators that generate its structure. We divide training between closed-book and open-book examples. Closed-book examples are finite and stable rules that should be memorized by the model, such as layer tags, layer families, module identifiers, and direction conventions, while open-book examples cover unbounded items like continuous values and statistics, which should be read from the runtime. The same distinction extends to tool-call training, where the model learns to parse returned objects instead of restating them from memory. We intend for the model to learn the operators that generate structure, not the specific values they produce.

Using these principles, we define a ten-step supervised training curriculum that gradually introduces more complex operators and tasks. Each stage builds on the previous one, starting with basic graph navigation and moving toward module comparison, path recovery, and enrichment analysis. We group similar stages in the prose; Table 2 presents the full curriculum.

Curriculum stages S0–S3 focus on foundational graph-navigation operators. S0 covers Ensembl-to-symbol and symbol-to-Ensembl conversions and resolution of ambiguous symbols, which are necessary for all downstream tasks. S1 introduces the lattice coordinate system, teaching the model to navigate the multiplex structure, retrieve layer information, and interpret rankings and metrics. S2 covers atomic graph facts and negative examples, such as edge existence, neighbors, layer membership, and degree. S3 introduces path-based and cross-layer operators, including connector and bridge nodes.

After foundational training, stages S4–S6 focus on the three structural views of the multiplex. S4 teaches the model to operate on bottom-up RWR-LOE neighborhoods, S5 trains it on top-down hierarchical modules, and S6 unifies these views through network connectivity. S5 also includes examples that teach correspondences between

Table 1: Model-facing RWR++ tool vocabulary. Related calls are grouped by function while preserving the tool names the model emits. The final column gives the curriculum step at which each group is introduced (Section 0.2.4).

Tools	Purpose	Primary inputs	Step
<code>get_neighbors;</code> <code>induce_subgraph</code>	Direct neighbors of a gene and the edges induced within a gene set	gene or gene set; layer scope	S2
<code>get_gene_layers;</code> <code>get_nodes_by_layer;</code> <code>get_layer_stats;</code> <code>get_component_summary</code>	Layer membership, presence of a set in a layer, per-layer statistics, and connected components	genes; layer identifiers	S2
<code>shortest_paths;</code> <code>get_path_layer_counts</code>	Shortest paths within and across layers, and the per-layer edge counts supporting a path	source and target genes; path	S3
<code>rwr;</code> <code>rwr_loe</code>	Random-walk expansion from seeds, and leave-one-out relatedness with its elbow cutoff	seed genes; query genes; top_k	S4
<code>get_rank;</code> <code>get_distance;</code> <code>get_spearman;</code> <code>get_pearson;</code> <code>get_dot_similarity</code>	Rank of a target gene from a seed, and pairwise distance or similarity under a stated metric	source and target genes; metric	S4
<code>get_rank_vector_summary;</code> <code>get_encoding_summary</code>	Compact summaries of rank-vector and encoding outputs without exposing dense vectors	seed or query genes; layer scope	S4
<code>get_seed_essentiality;</code> <code>get_layer_ablation;</code> <code>get_node_perturbation</code>	Effect of removing a seed, a layer, or a node on the resulting neighborhood	seed genes; layers; genes	S4, S6
<code>get_clade;</code> <code>get_lca;</code> <code>get_subtree;</code> <code>get_clade_relations</code>	Clade membership, lowest common ancestor of a gene pair, the subtree induced by a cut, and parent, child, and sibling relations	clade or module identifier; gene pair; cut height	S5
<code>rwr_netstats</code>	Module and cohesion statistics, with an empirical p -value where applicable	module identifiers; statistic; null size	S5, S6
<code>query_mygene;</code> <code>enrich_gene_set</code>	Gene annotation, and functional or module enrichment of a gene set	query or gene set; against; thresholds	S7

the RWR-LOE and MENTOR-EV views, preventing them from being learned as unrelated skills.

Stages S7 and S8 integrate the lattice structure by adding projection and enrichment tasks. S7 teaches the projection workflow, which combines the top-down and bottom-up views of the lattice. A projection neighborhood is selected, which can be a GWAS-derived candidate list, an RWR-LOE neighborhood around a single target, or any other relevant set of genes. The runtime then enriches that neighborhood against every level of the MENTOR-EV hierarchy, yielding an enrichment score and significance value for each clade. High enrichment across several distant clades is evidence of a single connected system, not unrelated biological processes. This reinforces the idea that tree distance does not mean that clades are unrelated (Section 0.2.3). Therefore, recovering bridge nodes, short cross-clade paths, and their supporting layers is essential. S8 teaches composition and context-ontology navigation in open-book mode. Given a supplied partial order, the model returns a context’s parents, children, and siblings, computes joins and ontology distances, and uses each join to name the corresponding union multiplex. It also learns to predict how neighborhoods, clades, and connectivity change when a layer is added or removed, which is crucial for understanding differences across sampled lattice points.

Finally, S9 consolidates the curriculum by combining earlier stages and preparing the checkpoint for the multistep reasoning required in Stage 2. The model trains on tool-call construction, selection, parsing, provenance, and task refusal, all of which are necessary for the mechanistic agent (Section 0.2.5) to operate effectively in the multiplex lattice.

The curriculum scales difficulty along several axes. We scale the lattice tier from single cell types to tissues, regions, and eventually the entire brain. Dendrogram cut height controls module size, progressing from coarse to fine-grained views of clades. We also scale operator complexity and book mode explicitly. To measure progress, we define task-specific gates that the model must pass before advancing to the next stage. All gates use held out question phrasings and graph regions for each family, as well as negative examples that measure abstention.

Corpus Generation and Validation

To build the training corpora for each curriculum stage, we generate questions procedurally from various points on the lattice. By sampling many contexts and layers, we allow the model to see each operator applied to many different networks, which encourages generalization. All targets have their ground truth produced by the deterministic runtime defined in Section 0.2.4, and examples are cached with their layer descriptor, metric, seed set, and graph version to ensure reproducibility. We freeze context, clade, and tuple assignments before generating training examples, then materialize each curriculum corpus on demand. Because assignments are frozen first, we can modify the sampling strategy or corpus volume without leaking information from the validation and test sets.

Every answer field is checked against a deterministic runtime, which ensures that both training examples and model outputs are valid. The runtime checks items such

Table 2: The ten-stage supervised curriculum. Book mode indicates whether targets are memorized (closed), read from supplied context (open), or produced by constructing and parsing a tool call. Data shares are provisional and will be reset once divergence validation establishes how strongly contexts differ across the lattice.

	Focus	Book mode	Share	Gate
S0	Entity normalization between symbols and Ensembl identifiers, including ambiguous symbols	closed	4%	Near-perfect accuracy on held out identifiers, and correct deferral on ambiguous symbols
S1	Lattice coordinate system: layer tags and families, module identifiers, and direction conventions	closed	4%	Near-perfect accuracy on held out tags and module identifiers
S2	Atomic graph facts: edge existence, neighbors, layer membership, degree and hubness, components, and calibration negatives	open	14%	Per-family accuracy on held out graph regions and phrasings, and correct refusal on calibration negatives
S3	Paths and cross-layer routes, path decomposition by layer, and connector and bridge nodes	open	8%	Path and bridge recovery on held out graph regions
S4	Bottom-up RWR-LOE neighborhoods: rank and score lookup, comparison, the elbow, leave-one-out support, and stability	open, tool	16%	Recovery and comparison accuracy stratified by distractor band
S5	Top-down MENTOR-EV hierarchy: clade membership, lowest common ancestor, subtree at a cut, and correspondence with RWR-LOE	open	16%	Hierarchy-navigation accuracy across cut heights, and correct cross-source containment
S6	Cross-clade connectivity: short paths between distant clades, bridge nodes, layer support, and cohesion against a null	open	12%	Cross-clade path recovery, and correct rejection of the false-separation reading
S7	The projection operation: select a seed neighborhood, enrich it against every hierarchy level, and return to the network	open, tool	12%	Correct projection, connectivity recovery, and interpretation, scored separately
S8	Composition and lattice scaling: open-book context-ontology navigation, joins, distances, and structural changes when layers are added or removed	open, tool	8%	Parent, child, and sibling set accuracy; join and ontology-distance accuracy; and correct structure on unions assembled from parts seen in training
S9	Tool-call consolidation: selection, argument construction, result parsing, provenance, and refusal of raw path arguments	tool	6%	Schema-valid tool-call rate and correct parsing on held out tools and phrasings

as path existence, module membership, and mechanistic claims, composing these checks based on the requirements of a task type. Using deterministic validation for our training examples and inference outputs allows cheap and scalable evaluation of reasoning throughout the process.

To provide sufficient training signal, our provisional lattice corpus includes 24 contexts and a full-brain multiplex. We expect to generate approximately 3 million supervised examples: roughly 10% closed-book, 75% open-book, and 15% tool-call examples, totaling about 1.2 billion training tokens. This corpus can be expanded to additional context as lattices for them become available, which we discuss further in

Section 0.4.3. Cached ground truth, not text, dominates storage, with an estimated 2–4 terabytes across the 24 contexts. This provides a second reason to materialize corpora in curriculum order instead of all at once. These estimates are provisional and will be adjusted based on results at each curriculum stage.

Splits and Leakage Control

Because contexts share genes, they also share edges, which creates a risk of leakage between training, validation, and test sets. To mitigate this risk, we hold out whole contexts, whole modules, and whole seed genes. Two to four of the 24 contexts will be held out entirely, so their hierarchies are never seen in training. Modules are held out by cutting the maximal subtrees at a chosen anchor height, since genes appear in every module above that height. All tuples sharing a seed gene go to the same split, and we limit the overlap between held out and training tuples. Additionally, each split is audited prior to training, allowing us to identify and remove any remaining leakage.

Using this protocol, we can reduce the risk of leakage while keeping biological structure intact. Contexts still share backbone edges, which are by definition present in every context. Instead of removing these edges, we bound and report them.

0.2.5 Stage 2: Mechanistic Agent

After training MENTOR-RL on all curriculum stages, we use the resulting checkpoint to initialize a mechanistic agent that explores the multiplex lattice and generates mechanistic interpretations. The agent is post-trained through preference optimization and reinforcement learning on long-horizon reasoning tasks. We formalize the agent loop and structured state in Section 0.2.5 and describe the task cases, corpora, and reward design in Sections 0.2.5–0.2.5. Section 0.2.6 details the full training pipeline and curriculum.

Agent Loop and Structured State

We model mechanistic exploration as a sequential decision process executed by a single policy with an internal self-verification step. The loop begins when the user provides a query q and optional evidence e , which may include free text, seed genes, or a multiplex. These initialize the working interpretation I_0 and the structured state S_0 .

Each step t includes multiple actions. First, given the current decision context, the agent emits a textual reasoning step r_t , naming its current subgoal or thinking trace, together with an optional tool call a_t . The tool call is executed by the deterministic runtime ε of Section 0.2.4 to produce an observation o_t ; when no call is made, the observation is empty. The same policy is then used for self-verification. Conditioned on the decision context and on its own reasoning, action, and observation, it updates

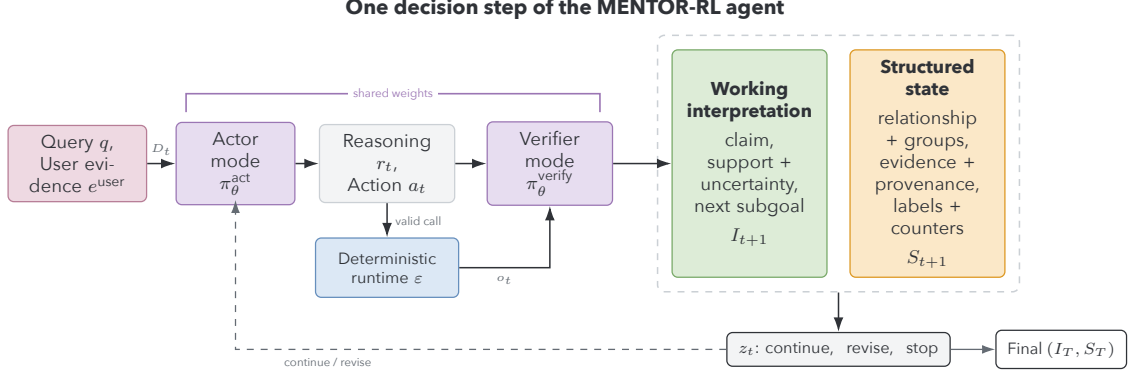


Figure 3: One iteration of the act–tool–verify loop. Actor and verifier modes share model weights, while the graph runtime remains deterministic. Verification produces synchronized human-readable and machine-readable views of the current hypothesis and a decision to continue, revise, or stop.

the interpretation and state and emits a continuation decision z_t of **continue**, **revise**, or **stop**. A decision of **revise** means the next iteration corrects the interpretation and state instead of extending them. One iteration is represented as

$$r_t, a_t \sim \pi_{\theta}^{\text{act}}(\cdot \mid D_t), \quad o_t = \varepsilon(a_t), \quad I_{t+1}, S_{t+1}, z_t \sim \pi_{\theta}^{\text{verify}}(\cdot \mid r_t, a_t, o_t, D_t),$$

where $D_t = (q, e, I_t, S_t)$ is the decision context and $\varepsilon(\emptyset) = \emptyset$ by convention. The actor and verifier are two prompting modes with the same weights, so biological vocabulary and tool-use competence learned in one mode transfer to the other. We acknowledge the risk that the verifier collapses into agreement with the actor, and will actively monitor this behavior during training. The loop repeats until the policy emits **stop** or the decision budget $T_{\max}$ is exhausted, and returns the pair (I_T, S_T) : a human-readable answer grounded in the accumulated evidence, and the machine-readable record used for scoring and provenance.

At each step, the working interpretation holds the current mechanistic claim, the main evidence supporting that claim, any uncertainty, conflict, or alternative explanation, and the next subgoal the model is trying to resolve in a human-readable markdown format. The structured state stores the same hypothesis in a machine-readable form suitable for tool execution, reward computation, and provenance tracking. It contains:

1. the user-provided anchors (q, e) ,
2. the current status of the working hypothesis: unknown, partial, validated, multiple, or insufficient support, where insufficient support indicates abstention,
3. the current findings: predicted groups, enriched clades, paths with their supporting layers, and bridge nodes, depending on the task requirements,

4. the evidence and provenance collected so far,
5. the current predicted mechanistic labels, each tied to the evidence that supports it,
6. the tool-use counters recording total and invalid calls, and
7. the remaining budget and continuation state, with the termination signal recorded at the end of the trajectory.

The findings that a task populates depend on the task type. Module tasks populate groups, while connectivity tasks populate paths, supporting layers, and bridge nodes, and projection tasks populate enriched clades together with the connectivity fields. Scoring depends only on the fields that are required by the metadata.

To ground the agent’s reasoning in evidence, we train three behaviors. The agent supports every claim with retrieved evidence, states its uncertainty explicitly, and abstains when support is weak. It uses the MENTOR-EV dendrogram modules as an index and confirms its hypotheses on the network, following the multiview structure described in Section 0.2.3. It returns both a human-readable interpretation and a structured state for record-keeping. The reward function and training curriculum reinforce these behaviors.

Task Cases and Targets

To train the agent, we define three categories of agent-training tasks that build on the lattice representation learned in Stage 1: module, connectivity, and projection tasks. Each has targets that provide deterministic rewards and support evaluation. These tasks combine to answer two kinds of questions. First, whether a set of genes forms a complete and coherent module, and second, how the modules connect to one another in the multiplex lattice. Each task type has targets that provide deterministic rewards and support evaluation.

Module tasks fall into four cases: partial-group completion, mechanism explanation, group refinement, and recognition of no shared mechanism. In partial-group completion, the agent receives a module with some genes held out and must recover the complete module. In mechanism explanation, the agent receives a complete module and must identify its supporting mechanistic evidence without adding or removing genes. In group refinement, the agent receives a module with distractor genes and must remove them to recover a mechanistically coherent group. In the no-shared-mechanism case, the agent receives a conflict-free set that is not mechanistically coherent and must report that the evidence provides insufficient support for a shared mechanism.

Connectivity tasks join the two module views through the network that underlies both. The agent is given two clades that are distant in the MENTOR-EV hierarchy and identifies a short path between them, along with the layers that support that

path. It also identifies any bridge nodes that connect the two modules and provides mechanistic evidence for their role. This task tests the agent’s ability to reason about the underlying multiplex structure and recover connectivity that is not captured by the bottom-up or top-down views alone.

Finally, we train the model on projection tasks, which require the agent to project a seed neighborhood onto the MENTOR-EV hierarchy and identify the clades that are enriched for that neighborhood. Because the neighborhood was built by propagation over the network, enrichment landing in several distant clades indicates one connected system, not unrelated processes, and the agent reads it that way. It then recovers the bridge nodes and short paths that connect those clades and provides mechanistic evidence for their role. Projection tasks are scored on three targets separately: the projection itself, the connectivity recovered to support it, and the interpretation placed on that connectivity. A candidate passes only when all three clear, since a correct projection paired with a failed connectivity recovery is precisely the error that the network view exists to prevent.

Each task family is used to organize training and scoring, but not the agent’s behavior. The agent never receives a task-type label, but during training, the scorer knows which task is sampled and reads the corresponding structured state fields to compute rewards. At deployment, the agent answers the query with whichever operators the question requires.

The agent curriculum culminates in the applied case, in which the projected seed is a drug target or a disease signal. Here the agent additionally applies directional and translational filtering. Directional filtering asks whether the candidate intervention opposes the disease-associated direction within a specific module instead of merely acting near it. Directionality comes from differential expression with respect to disease context, which is returned by the runtime. Translational filtering ranks surviving candidates based on multiple metrics. Evidence strength is computed from retrieved provenance, while adverse-event risk, brain penetration, feasibility, and testability are reported for expert review and do not enter the training reward.

Module tasks use scorer-only membership targets drawn from RWR-LOE neighborhoods and MENTOR-EV modules. Recovery and refinement target the complete module, explanation preserves the supplied module, and `none` has an empty target. Because these computational modules usually lack per-entry experimental annotations, explanations are scored against evidence retrieved during exploration, not a hidden biological label.

The runtime supplies family-specific targets for connectivity and projection: optimal paths, connector and bridge nodes, and supporting layers for connectivity; and per-clade enrichment, cross-clade connectivity, and allowed or disallowed interpretations for projection. Any optimal path is accepted. All targets remain hidden from the agent, and evidence grounding is scored consistently across task families.

Projection, return to network, and the applied case

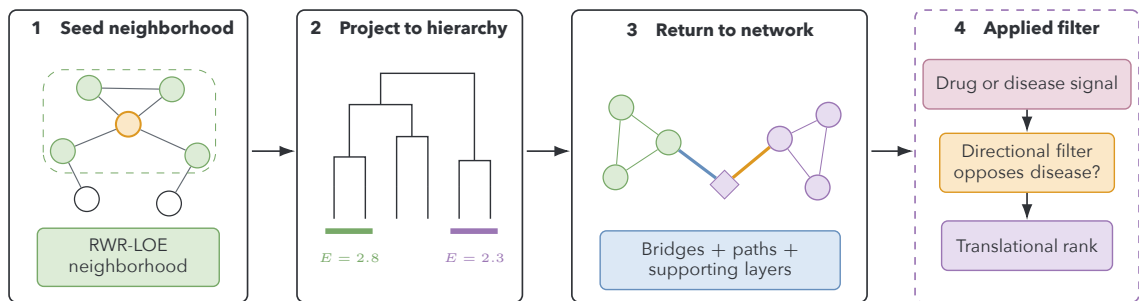


Figure 4: Projection and return to network, with the applied drug-target or disease-signal extension. A seed neighborhood is enriched against the hierarchy; significant, potentially distant clades are then connected and interpreted on the multiplex. Directional and translational filters are applied only after this network confirmation.

Task Corpora

We generate module tasks from top-down MENTOR-EV modules and bottom-up RWR-LOE neighborhoods sampled across the lattice. For each module of size n , we deterministically materialize up to four instances: **explanation** uses the full module, **recovery** drops k_{drop} genes, **refinement** adds k_{add} distractors, and **none** constructs an unrelated pseudo-module with an empty target. Perturbation size increases with task difficulty and module size.

The two module sources differ only in where distractors come from. For MENTOR-EV modules, noise and **none** genes are drawn from difficulty-modulated *dendrogram distance bands*, defined by walking up the module’s ancestry in the tree, so that difficulty controls how topologically close the distractors are to the module. For RWR-LOE modules we substitute rank bands, drawing from ranks just beyond, moderately beyond, or far beyond the seed’s elbow cutoff. In both cases **none** genes are additionally constrained to be conflict-free, so that the sampled genes do not form a coherent module of their own, and all selection is deterministic given a fixed seed, making the corpus reproducible. Module-size bins are assigned after construction from the realized module sizes, and all splits follow the protocol of Section 0.2.4.

Connectivity instances pair sampled clades at specified tree distances; projection instances sample a seed neighborhood from the sources in Section 0.2.4, with difficulty set by how far apart the enriched clades fall in the hierarchy. The runtime then materializes the path, layer support, enrichment, and connectivity targets defined in Section 0.2.5.

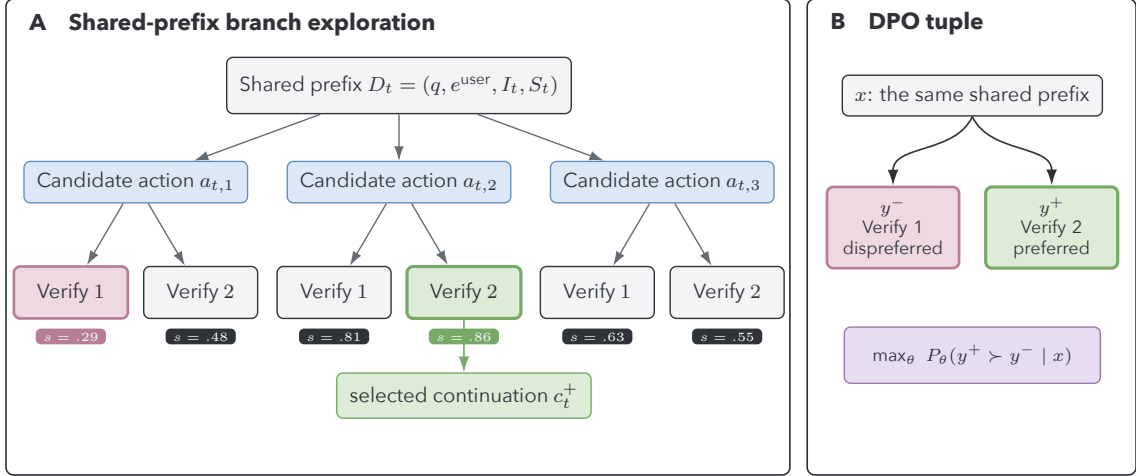


Figure 5: Shared-prefix branch exploration and DPO preference construction. Candidate actions and verifier updates are compared only after the same decision context. Task-quality selection chooses the retained continuation; eligible lower-scoring branches from that pool become dispreferred continuations when the normalized margin exceeds τ .

Trajectory Generation

Trajectories for reinforcement learning and preference optimization are generated by shared-prefix branch exploration. At each decision point, the policy proposes several candidate continuations from the same state, the runtime executes any valid tool calls among them, and the resulting branches are scored against one another. One branch is retained as the trajectory, while eligible alternatives become material for preference pairs. The procedure is described below.

Given a sampled task and initialized state, we sample multiple action candidates $(r_{t,i}, a_{t,i})$ from the current decision context D_t and execute valid tool calls in the deterministic graph runtime. To ensure that candidates with the same prefix explore distinct regions of the action space, we adapt verbalized sampling to the visible task framing [55]. For example, a query asking the agent to recover missing members biases candidates toward random-walk expansion, neighborhood expansion, subgraph validation, or a direct decision. Framing affects which actions are explored while the underlying preference rewards do not vary. When a sampling round for recovery or refinement fails to produce valid tool calls, we issue a tool-coverage retry that requires a valid graph operator whenever one can be formed, preventing the branch pool from degenerating into tool-free continuations. Conditioned on each candidate action and observation, the same policy samples multiple verification candidates, yielding a tree of candidate next states rooted at a shared prefix. Figure 5 visualizes this branching scheme.

Each candidate branch is assigned a deterministic local score computed from the structured state, runtime outputs, and hidden targets for its task family. These local scores are used to rank same-prefix candidates and mine DPO preference pairs against that selection. Because several branches tie frequently or fall within a small margin of each other, continuation selection accepts any candidate within a margin ϵ of the best normalized score. Ties between candidates are broken with task-specific heuristics, such as prompted seed-gene retention and tool-supported evidence, which favor well-grounded candidates over those that score highly by chance. We defer the formal score definitions to Section 0.2.5.

The preferred candidate is the branch selected as the trajectory continuation, not the strict score maximizer, since selection resolves near-ties using task-specific heuristics and therefore reflects the move judged best for the task. Candidates within a margin τ of the selected branch are discarded so that no pair is built from a near tie, and the remainder form the dispreferred pool. From that pool we draw examples across **easy**, **medium**, and **hard** difficulty bins, taking the lowest, median, and highest scoring remaining candidates respectively. Each pair is stored with its bin, task type, decision step, scores, and margin, and the corpus is rebalanced across task types and bins so that training is not dominated by the categories that happen to yield the most informative pairs.

Each retained trajectory is also logged as a sequence of turn records, from which we render short per-step findings and a final mechanistic summary. These become supervised examples and preference pairs for the training pipeline described in Section 0.2.6.

Model prompts carry a compact summary of the current state, so context does not grow as the trajectory lengthens. Each prompt supplies the current working interpretation, the relationship status, the predicted groups, the mechanistic labels tied to the evidence handles that support them, the tool and evidence counters, and the current observation. Prior observations are not appended, so prompt length stays bounded as trajectories grow, allowing for long-horizon exploration that is not limited by memory. The full evidence payloads are retained in the logged trajectory artifacts, where they remain available for scoring, audit, preference-pair mining, and expert review.

We provide several query modes and evidence types to represent the diversity of real-world inputs. During training, we sample queries from task-specific templates paraphrased by an open-weight model to ensure linguistic diversity. The agent receives either minimal evidence, consisting only of the query and associated seed genes, or graph-supported evidence, which includes a relevant multiplex subgraph. At inference time, the agent receives a natural-language query with optional graph or seed evidence. However, it does not see the task-type label, the true relationship status, or the hidden targets, which are reserved for scoring and trajectory generation.

We define a three-stage curriculum for the mechanistic agent, which gradually increases in task complexity and trajectory length. The first stage focuses on module tasks, where the agent learns to recover and refine modules from partial

or noisy inputs. The second stage introduces connectivity tasks, where the agent identifies paths and bridge nodes between distant clades. The final stage focuses on projection tasks, where the agent projects neighborhoods onto the hierarchy and recovers mechanistic evidence for the resulting connections, synthesizing the skills learned in the previous stages. Each stage builds on the previous one, ensuring that the agent develops a comprehensive understanding of the multiplex structure and the mechanistic relationships within it.

Reward Design

We define training-time scores over structured state transitions, deterministic runtime metadata, and hidden scorer-only targets specific to each task family; natural-language reasoning is not scored directly. This keeps scoring reproducible, machine-parseable, and scalable without relying on human judges during training. Every score has four shared components: schema compliance, task correctness, evidence-grounded mechanistic quality, and efficiency.

Task correctness is computed the same way for all task types, but the underlying targets differ. Each family declares which findings in the structured state are scored and against which hidden target, and each declared component is scored by overlap with that target. For families with several scored components, we compute the task score as a weighted mean, so $q^{\text{task}}(S) \in [0, 1]$ regardless of family.

Because DPO compares continuations after the same prefix, we score a candidate branch b by its change in task correctness and mechanistic quality:

$$s_{t,b}^{\text{local}} = w_s s_{t,b}^{\text{schema}} + w_q \Delta q_{t,b}^{\text{task}} + w_m \Delta m_{t,b} - w_e p_{t,b}^{\text{call}} - p_{t,b}^{\text{guard}}. \quad (1)$$

Here, $q^{\text{task}}(S) \in [0, 1]$ is the correctness score associated with the sampled task family, $\Delta q_{t,b}^{\text{task}}$ is the difference between q^{task} at t and $t + 1$, and $\Delta m_{t,b}$ is the difference between the mechanistic quality at t and $t + 1$. The schema term is equal to one only when the candidate state and any tool request satisfy their typed validators. The guard penalty covers task-incompatible behavior and varies by task family. For module tasks, an **explanation** branch is penalized for removing members from a module that is already complete. Across all families, mechanistic improvement is clamped to be non-positive whenever a branch degrades task correctness.

For recovery, refinement, and explanation tasks, task correctness is the best match between any predicted group and the hidden module target C :

$$q^{\text{module}}(S; C, C_{\text{type}}) = \max_{G \in \mathcal{G}(S)} [\alpha J(G, C) + \beta P(G, C) + \gamma R(G, C)],$$

where $\mathcal{G}(S)$ is the set of predicted groups in structured state S , with an empty group used when no prediction is present, and J , P , and R representing the Jaccard similarity, precision, and recall of a predicted group against the target. The weights α , β , and γ sum to one and are set per task case: recovery upweights recall, refinement upweights precision, and explanation weights the three equally. Final values will be

set during development. This formulation supports a multiple-groups state while retaining a single-module hidden target. For module tasks, q^{module} takes the role of q^{task} in Equation 1.

For a **none** task, the correctness score instead rewards calibrated abstention:

$$q^{\text{none}}(S) = 0.6 \mathbf{1}[\text{status}(S) = \text{insufficient_support}] + 0.4 \mathbf{1}[\mathcal{G}(S) = \emptyset], \quad (2)$$

where $\text{status}(S)$ is the hypothesis status of the structured state (Section 0.2.5) and $\mathbf{1}[\cdot]$ equals one when its condition holds and zero otherwise.

Thus, **none** is an instantiation of the shared task-correctness term, not a separate reward pathway. The multiple-groups status remains a supported state, but it receives no separate score unless the corpus later materializes a corresponding task family.

For connectivity tasks, q^{conn} combines path validity and optimality, connector and bridge-node recovery, and supporting-layer recovery. If v_{path} denotes the fraction of returned paths that are valid and optimally short, B the returned connector and bridge nodes, and L the returned supporting layers, then

$$q^{\text{conn}}(S) = \eta_p v_{\text{path}}(S) + \eta_b J(B(S), B^*) + \eta_l J(L(S), L^*), \quad \eta_p + \eta_b + \eta_l = 1, \quad (3)$$

with component weights η_p , η_b , and η_l summing to one and set during development.

Any optimal path is accepted. For projection tasks, the correctness score preserves the operation’s three distinct targets:

$$q^{\text{proj}}(S) = \eta_e q^{\text{enrich}}(S) + \eta_c q^{\text{conn}}(S) + \eta_i q^{\text{interp}}(S), \quad \eta_e + \eta_c + \eta_i = 1. \quad (4)$$

The enrichment component scores significant clade recovery, where the runtime computes significance with a one-tailed Fisher exact test, and copied runtime values. The connectivity component scores the return to the network, and the interpretation component scores the structured allowed and disallowed readings. These components provide dense local feedback, but a completed projection passes only when all three clear their individual thresholds, which are set during development, so projection accuracy cannot compensate for failed connectivity recovery. Directional and translational fields in the applied case are scored within the structured interpretation component when present.

We define mechanistic quality $m(S) \in [0, 1]$ from the evidence the agent has retrieved. Each component measures the alignment between the mechanistic labels in the structured state and the evidence that supports them. Let g denote grounding, or the fraction of asserted entities and labels that appear in retrieved observations. Let n denote network support, or the fraction of asserted relationships confirmed in the multiplex in stated layers. Finally, let c denote corroboration, or the fraction of claims supported by at least two independent sources among enrichment, annotation, and network. For a structured state that asserts mechanistic labels, we define

$$m^{\text{claim}} = w_g g + w_n n + w_c c,$$

with weights summing to one and set during development. For an abstaining state, or one whose status is `insufficient_support` (Section 0.2.5), we set $m^{\text{abs}} = w_u u + w'_g g$, where u is one minus the best corroboration available for any candidate claim, so withholding is rewarded in proportion to the weakness of the evidence. On a non-`none` task, this score is forced to zero when a target is recoverable, preventing unsupported abstention from earning mechanistic reward.

The local efficiency term penalizes the fraction of a branch’s tool calls that are invalid, scaled by a hyperparameter λ_{inv} . A branch that makes no tool calls receives no penalty.

Schema-invalid requests, exact duplicates, missing observations, and error observations count as invalid; a branch that makes no tool call receives no call penalty. Trajectory length is omitted locally because it is constant across same-prefix candidates and therefore cannot affect their DPO ranking.

Terminal Reward Decomposition for GRPO

Unlike DPO, which compares local same-prefix branches, policy gradient methods require a scalar reward for a completed rollout. We therefore define a terminal reward over the final structured state and trajectory log:

$$\begin{aligned} R_T = & w_s^T s_{\text{schema}}^T + w_q^T q_T^{\text{task}} + w_{\Delta q}^T (q_T^{\text{task}} - q_0^{\text{task}}) \\ & + w_m^T m_T + w_{\Delta m}^T (m_T - m_0) - w_e^T p_{\text{eff}}^T. \end{aligned}$$

Here, q_T^{task} uses the same family-specific definition as local scoring, so absolute and cumulative correctness apply equally to module, `none`, connectivity, and projection tasks. The terminal mechanistic terms use the final evidence-grounded score and its change from initialization. Mechanistic credit is capped when the task-specific correctness gate fails, preventing a fluent interpretation from compensating for an incorrect module, path, or projection. The terminal schema score additionally requires a valid final state and a recorded termination mode, whether the agent stops voluntarily or exhausts its budget.

The terminal efficiency penalty aggregates trajectory length and invalid tool use across the rollout:

$$p_{\text{eff}}^T = \lambda_1 \frac{T}{T_{\text{max}}} + \lambda_2 \frac{\sum_{t=0}^{T-1} n_{\text{inv}}^t}{\max\left(1, \sum_{t=0}^{T-1} n_{\text{call}}^t\right)}. \quad (5)$$

The denominator convention makes the invalid-call term zero for a trajectory that makes no tool calls. The local score omits trajectory length because the same-prefix candidates have all reached the same point; complete rollouts, however, may vary in length, so the terminal reward penalizes verbosity through T/T_{max} .

The local scores above supply the preference signal for DPO, while the terminal reward supplies the scalar return optimized during GRPO.

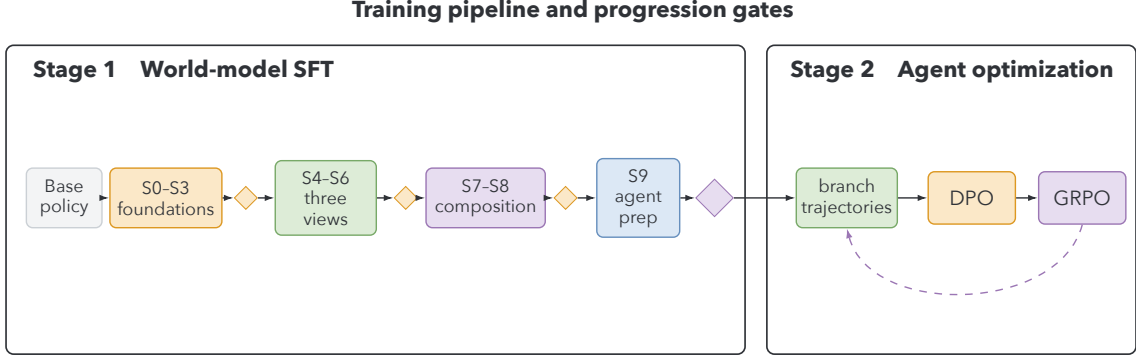


Figure 6: The gated two-stage training pipeline. Stage 1 advances through supervised curriculum steps only after held out gates clear. Its checkpoint initializes trajectory generation and preference optimization in Stage 2, followed by online GRPO. Checkpoint evidence accumulates from structural evaluations through agent behavior and expert review.

0.2.6 Training Pipeline and Curriculum

We first train the policy with SFT using the curriculum in Section 0.2.4. Training will run on OLCF’s Frontier supercomputer [3], with Slurm [54] for job scheduling, Hugging Face and TRL [52] for optimization, DeepSpeed ZeRO-3 [39] for model parallelism, Low-Rank Adaptation for parameter-efficient fine-tuning [18], and Weights & Biases [8] for convergence tracking. We use gpt-oss-120b [34] as the base policy and early-stopping to prevent overtraining.

After SFT on all curriculum stages, the trajectory method in Section 0.2.5 generates preference pairs scored by Section 0.2.5 to supply data for the next stage. The selected continuation is c_t^+ , and eligible alternatives across the **easy**, **medium**, and **hard** bins supply c_t^- . DPO increases the likelihood of c_t^+ relative to c_t^- under a frozen reference policy without a separate reward model [38]. We regenerate trajectories from updated checkpoints as training progresses.

The DPO checkpoint then initializes GRPO for long-horizon exploration. Each rollout receives an advantage equal to its terminal reward standardized within its rollout group, removing the need for annotated trajectories or a learned value model [43]. The DPO checkpoint remains the fixed reference policy, anchoring online exploration while GRPO promotes trajectories beyond the logged preference data.

We monitor tool-use entropy, trajectory diversity, and the single-pass RWR shortcut, in which one expansion recovers much of a module without meaningful exploration. We monitor single-pass shortcuts because they can lead to task correctness without the multi-step reasoning that rewards are meant to encourage. If shortcuts emerge, we introduce adversarial modules or a diversity bonus; these monitors must clear before advancement.

Each transition is gated. Stage 1 uses the per-step gates in Table 2; DPO must improve module recovery and mechanistic grounding over SFT on held out splits; and GRPO must improve the flagship evaluations. Deployment additionally requires expert preference to rise across the base, SFT, DPO, and GRPO checkpoints under Section 0.2.7.

Difficulty scales throughout optimization. Stage 1 independently advances book mode, operator complexity, lattice tier, cut-height granularity, and distractor band over curriculum stages. DPO moves from **easy** to **hard** pairs and from module recovery and refinement to connectivity and projection. GRPO increases the horizon $T_{\max}$, realized module size, perturbation size, distractor proximity, and lattice tier up to higher-order brain combinations. Batches remain balanced across task types, difficulty bins, and both module sources.

0.2.7 Evaluation

Stage 1 Evaluation: World Model SFT

Stage 1 advances through the gates in Table 2, which are evaluated on held out data at each step. Two flagship evaluations then measure mechanistic reasoning. The first generates a hierarchy for a context held out from training, and the second projects a seed set onto that hierarchy, returning it to the network. For both flagships, we use probes and interventions to test whether the recovered structure is causal.

In the first flagship, the model receives the active layers of an unseen context along with a query gene set, and it returns pairwise co-membership, lowest common ancestor depth, and a partition of the query gene set into clades. We score these results against a ground-truth dendrogram from the runtime. Because hierarchies diverge across contexts, this flagship tests whether the model has internalized the lattice structure.

The second flagship follows the projection described in Section 0.2.5. A seed neighborhood is selected, then enriched against the hierarchy, and the model recovers the paths and bridge nodes connecting significantly enriched clades. This flagship tests whether the model can unify multiplex views into a single interpretation.

To test whether lattice operators are internalized, we use multilayer-perceptron *probes* [2], which are classifiers trained on hidden layer activations, to recover edge presence, layer membership, clade co-membership, and connector status. We then intervene on those activations and measure whether answers change, distinguishing causal structure from mere decodability. This is a gate rather than a diagnostic and is repeated under the lean-context condition without the full rank-vector legend. Table 3 summarizes both flagships and the probe gate.

Stage 2 Evaluation: Mechanistic Agent DPO and GRPO

We will calculate evaluation metrics for Stage 2 by parsing the model’s terminal structured state S_T , enabling scalable checkpoint evaluation. All evaluations will use a

Table 3: Scoring for the Stage 1 flagships and the cross-cutting probe gate. Components are reported separately; projection passes only when projection, connectivity recovery, and interpretation all clear their thresholds.

Evaluation or gate	Component	Metric
Held-out context hierarchy generation	Partition agreement	Adjusted Rand index [19], which measures agreement between two partitions corrected for chance occurrence
	Clade co-membership	Same-clade accuracy over query gene pairs
	Ancestry	Lowest common ancestor depth accuracy per gene pair
Projection and return to network	Projection	Jaccard of the predicted significant clades against the runtime enrichment, with per-clade score and q -value copy accuracy
	Connectivity recovery	Jaccard of recovered connector and bridge nodes, path validity rate, and the fraction of recovered paths at optimal length
	Interpretation	Accuracy of the connected-system reading, and the rate at which the disallowed reading that tree distance implies separation is asserted
Probing and intervention gate	Decodability	Probe classification accuracy for edge presence, layer membership, clade co-membership, and connector status
	Causality	Change in the model’s answer when the probed activation is modulated to flip its predicted value
	Scaffold independence	Probe and interpretation accuracy retained under the lean context condition, without the full rank-vector legend

held out set of MENTOR-EV- and RWR-LOE-derived modules from the brain lattice. We will measure module, enriched-clade, and cross-clade path recovery with Jaccard, precision, recall, and F1. Jaccard penalizes both missing and extra genes, precision and recall separate those error directions, and F1 provides a balanced summary. We will also report mechanistic-grounding quality to test whether claims align with the evidence retrieved by the model.

We will report the weighted composite reward R_T , defined in Section 0.2.5, to compare reward magnitude on validation and holdout splits. For the **none** task type, where $C = \emptyset$, we will parse S_T for the terminal hypothesis status, $\text{status}(S_T)$. In addition to global accuracy and reward, we will distinguish the two error directions: claiming that no mechanism exists for a related gene set (false negative) and fabricating a mechanism for unrelated entities (false positive). We

will also report mean trajectory length, budget utilization, and invalid-tool-call rate, enabling comparison of holdout and validation efficiency under Equation 5.

We will stratify module-task results by module size, noise, task type, and module source to examine the effect of curriculum staging. We will also stratify by lattice tier and cut-height granularity to measure performance across regions of the lattice. Each axis will be divided into three discrete bins. We will also assess stability by re-running the runtime’s enrichment and propagation on leave-one-out subsets of each seed set and reporting the mean Jaccard between the resulting supporting-evidence sets. These metrics will show how MENTOR-RL performs as difficulty increases and task type changes; comparison by module source will reveal whether the model generalizes across construction methods or favors one module type.

Alongside empirical validation, we will conduct blinded comparative evaluations of domain experts’ model preferences. First, we will use an inter-model preference test to compare interpretation outputs from a panel of frontier and open-weight baselines. At present, these models would be MENTOR-RL, GPT-5 [45], LLaMA 4 [29], Kimi K2 [48], and Nemotron 3 Super [33], but they are subject to change with new releases. Between 5 and 15 domain experts will receive paired samples containing a MENTOR-RL output and an output from another randomly selected model. We will report the proportion of pairs in which experts prefer MENTOR-RL. A subset of pairs will be shared across raters to measure inter-expert reliability.

We will then use the same protocol for intra-model preference testing across the base, SFT, DPO, and GRPO-trained MENTOR-RL checkpoints. These blinded comparisons will assess characteristics such as style, tone, and concision that empirical metrics do not capture easily.

Ablation Studies

To ensure that each component of our methodology contributes to an improvement in mechanistic interpretation, we will ablate both training stages and design elements. All ablations will be evaluated using the holdout metrics defined in Section 0.2.7.

We ablate phases of the training pipeline to test whether each stage improves on the preceding checkpoint. First, we compare the SFT-only checkpoint with the SFT-and-DPO checkpoint to test whether preference optimization improves interpretation. Next, we compare the DPO checkpoint with the full pipeline to determine whether long-horizon RL improves on local preference training. Finally, we ablate Stage 1 entirely, training the agent from the base policy without the world-model curriculum and comparing it with the complete pipeline on module-recovery, connectivity, and projection tasks. This ablation tests the proposal’s central claim that the world model is necessary rather than merely helpful. Preliminary trajectory diagnostics in Section 0.4.1 indicate what to expect. An untrained policy prunes distractors successfully but fails to recover held out members or ground its claims in retrieved evidence.

We also ablate architectural and reward decisions. First, we remove the policy’s verifier mode to test whether self-verification improves results. Second, we remove the efficiency penalty p^{eff} to test whether MENTOR-RL learns reasonable trajectory lengths without regularization. Finally, we remove progressive difficulty bins and train on all bins simultaneously to test whether the model can learn from hard examples without curriculum staging.

0.3 Expected Results

0.3.1 Expected Empirical Results

We expect MENTOR-RL to recover modules and cross-module paths accurately in held out lattice contexts, measured by Jaccard, precision, recall, and F1; project seed neighborhoods to enriched clades and connecting paths; and ground interpretations in retrieved evidence. Probes should decode lattice structure, and interventions should show that it is causal to reasoning. Improvement across SFT, DPO, and GRPO checkpoints on unseen contexts would indicate out-of-distribution generalization.

0.3.2 Expected Behavioral Results

We expect the act–verify loop to reduce invalid calls and unsupported claims, the efficiency penalty to shorten trajectories without reducing recovery, and expert preference to rise across the base, SFT, DPO, and GRPO checkpoints. The corresponding verifier, efficiency, and checkpoint ablations in Section 0.2.7 test each claim.

0.3.3 Deliverables

We will release, under licenses compatible with the underlying data sources, the following artifacts:

1. the lattice construction code and context ontology,
2. the lattice-grounded training environment, including the `ctypes` multiplex runtime, tool vocabulary, and deterministic scoring functions,
3. train, validation, and test split indices at the context, subtree, and tuple level, a leakage audit, and the code required to reconstruct the raw training corpus and cached API responses,
4. trained MENTOR-RL checkpoints for each stage of the pipeline (SFT stages S0-S9, DPO, and GRPO),
5. the flagship harnesses, including held out contexts and reserved target genes, and

6. training and generation code, curriculum configurations, and logged trajectories to support reproducible re-training and ablation.

By releasing a deterministically-scored, open-source mechanistic exploration world model and agent together with its training environment, we aim to lower the cost of biological interpretation research and establish a reproducible benchmark that does not require proprietary model access. The scoring framework is domain-specific, but the training process to learn operators over networks in a lattice structure is transferable to other scientific settings with curated ground truth.

0.4 Discussion

0.4.1 Preliminary Results

Several preliminary results support the feasibility of the proposed approach. First, we demonstrated full fine-tuning of the 120-billion-parameter open-weight gpt-oss-120b model [34] using data, tensor, and expert parallelism on OLCF’s Frontier supercomputer. We have trained on up to 128 nodes. This infrastructure, together with the ability to test smaller models such as gpt-oss-20b under the same training and evaluation pipeline, meets the capacity requirements for the multiplex-lattice dataset.

We also implemented the Stage 2 trajectory-generation loop (Section 0.2.5) using the same gpt-oss-120b model. The loop runs end-to-end, provides a dashboard for run inspection, and produced no parsing errors in the preliminary runs. However, initializing trajectory generation from a base model without Stage 1 training yields poor performance. The protocol succeeds on refinement tasks but fails on recovery tasks and provides weak mechanistic grounding. These results support the need for Stage 1 to provide domain knowledge and tool-calling capabilities before Stage 2. Because they are based on a small number of runs and do not include the full evaluation suite, we will evaluate the loop more thoroughly after Stage 1 training.

Finally, we generated the corpora for the Stage 1 S0 and S1 curriculum steps. Our current focus is S0 training, which covers Ensembl-to-symbol and symbol-to-Ensembl mapping, as well as ambiguous-symbol resolution.

To begin Stage 1 training, we tested four tokenization approaches for mapping tasks. First, we used the gpt-oss-120b base tokenizer, fitted to its pretraining corpus with byte-pair encoding (BPE), as a control. Second, we fit a BPE tokenizer only to Ensembl identifiers and gene symbols, then concatenated the resulting vocabulary with the base vocabulary; we call this **domain-BPE** tokenization. Third, we encoded the Ensembl prefix (ENSG) as an atomic token, fit BPE to the remaining identifier suffixes and gene symbols, and concatenated that vocabulary with the base vocabulary; we call this **atomic+domain-BPE** tokenization. Finally, we represented each Ensembl identifier using atomic tokens for its prefix and accession

body, represented each gene symbol with one atomic token, and tokenized module identifiers compositionally; we call this **atomic** tokenization.

Our preliminary results show that base tokenization performs poorly on mapping tasks. We trained gpt-oss-120b on approximately 2,000 questions from a small S0 subset under each tokenization approach and evaluated it on 1,000 hard-subset questions. Overall accuracy increased from 45.6% with the base tokenizer to 85.9% with atomic tokenization. The mapping-specific gains were larger. Symbol-to-Ensembl accuracy increased from 8.3% to 100%, and Ensembl-to-symbol accuracy increased from 5.6% to 100%. These results select atomic tokenization for S0, pending confirmation at full vocabulary scale with held out and low-frequency entities.

0.4.2 Plan of Work

We will complete S0–S9 sequentially, materializing each corpus after the preceding gate clears; S0 is underway. The resulting Stage 1 checkpoint will undergo the two flagship evaluations, the cross-cutting probe gate, and curriculum ablations before generating Stage 2 trajectories. DPO on those trajectories will initialize GRPO, followed by the final evaluations, ablations, and deliverables in Section 0.3.3.

0.4.3 Limitations and Future Work

We identify several limitations and possible extensions. First, the lattice structure depends on the assumption that different contexts generate meaningfully different hierarchies. We will validate this assumption by comparing hierarchies across contexts; insufficient divergence would limit the lattice’s utility. The approach also requires substantial compute to construct the RWR-LOE and MENTOR-EV corpora and train the model on the multiplex-lattice dataset. Although we have access to OLCF’s Frontier supercomputer, these requirements may limit accessibility for other researchers.

Future work can extend the lattice to other organ systems and disease states, yielding a more comprehensive model checkpoint. Extending the lattice across species could support cross-species mechanistic reasoning. Additional evidence modes and curriculum stages could also teach more operators and support more complex reasoning across modalities.

Bibliography

- [1] Shibla Abdulla et al. “CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data”. In: *Nucleic Acids Research* 53.D1 (2025), pp. D886–D900.
- [2] Guillaume Alain and Yoshua Bengio. “Understanding intermediate layers using linear classifier probes”. In: *arXiv preprint arXiv:1610.01644* (2016).
- [3] Scott Atchley et al. “Frontier: Exploring Exascale”. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC ’23. Denver, CO, USA: Association for Computing Machinery, 2023. ISBN: 9798400701092. DOI: [10.1145/3581784.3607089](https://doi.org/10.1145/3581784.3607089). URL: <https://doi.org/10.1145/3581784.3607089>.
- [4] Albert-László Barabási, Natali Gulbahce, and Joseph Loscalzo. “Network medicine: a network-based approach to human disease”. In: *Nature reviews genetics* 12.1 (2011), pp. 56–68.
- [5] Albert-Laszlo Barabasi and Zoltan N Oltvai. “Network biology: understanding the cell’s functional organization”. In: *Nature reviews genetics* 5.2 (2004), pp. 101–113.
- [6] Sumanta Basu et al. “Iterative random forests to discover predictive and stable high-order interactions”. In: *Proceedings of the National Academy of Sciences* 115.8 (2018), pp. 1943–1948.
- [7] Bgee Group. *HsapDv: Human Developmental Stages Ontology*. OBO Foundry. <http://purl.obolibrary.org/obo/hsapdv.owl>. 2024.
- [8] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: <https://www.wandb.com/>.
- [9] Katy Börner et al. “Anatomical structures, cell types and biomarkers of the Human Reference Atlas”. In: *Nature Cell Biology* 23.11 (2021), pp. 1117–1128.
- [10] Yiqun Chen and James Zou. “Simple and effective embedding model for single-cell biology built from ChatGPT”. In: *Nature biomedical engineering* 9.4 (2025), pp. 483–493.
- [11] Paul F Christiano et al. “Deep reinforcement learning from human preferences”. In: *Advances in neural information processing systems* 30 (2017).

- [12] Ashley Cliff et al. “A high-performance computing implementation of iterative random forest for the creation of predictive expression networks”. In: *Genes* 10.12 (2019), p. 996.
- [13] Lenore Cowen et al. “Network propagation: a universal amplifier of genetic associations”. In: *Nature Reviews Genetics* 18.9 (2017), pp. 551–562.
- [14] Manlio De Domenico et al. “Mathematical formulation of multilayer networks”. In: *Physical Review X* 3.4 (2013), p. 041022.
- [15] Alexander D Diehl et al. “The Cell Ontology 2016: enhanced content, modularization, and ontology interoperability”. In: *Journal of Biomedical Semantics* 7 (2016), p. 44.
- [16] Daya Guo et al. “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning”. In: *arXiv preprint arXiv:2501.12948* (2025).
- [17] Leland H Hartwell et al. “From molecular to modular cell biology”. In: *Nature* 402.Suppl 6761 (1999), pp. C47–C52.
- [18] Edward J Hu et al. “Lora: Low-rank adaptation of large language models.” In: *Iclr* 1.2 (2022), p. 3.
- [19] Lawrence Hubert and Phipps Arabie. “Comparing partitions”. In: *Journal of Classification* 2.1 (1985), pp. 193–218.
- [20] ISO. *ISO/IEC 14882:2011 Information technology — Programming languages — C++*. Geneva, Switzerland: International Organization for Standardization, Feb. 2012. URL: <http://www.iso.org>.
- [21] David Kainer et al. “RWRtoolkit: multi-omic network analysis using random walks on multiplex networks in any species”. In: *GigaScience* 14 (2025), gfa028.
- [22] Chan Yeong Kim et al. “HumanNet v3: an improved database of human gene networks for disease research”. In: *Nucleic Acids Research* 50.D1 (2022), pp. D632–D639.
- [23] Sebastian Köhler et al. “Walking the interactome for prioritization of candidate disease genes”. In: *The American Journal of Human Genetics* 82.4 (2008), pp. 949–958.
- [24] Takeshi Kojima et al. “Large language models are zero-shot reasoners”. In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.
- [25] Kenneth Li et al. “Emergent world representations: Exploring a sequence model trained on a synthetic task”. In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: [2210.13382](https://arxiv.org/abs/2210.13382) [cs.LG].
- [26] Pan Lu et al. “Eubiota: Modular Agentic AI for Autonomous Discovery in the Gut Microbiome”. In: *bioRxiv* (2026). DOI: [10.64898/2026.02.27.708412](https://doi.org/10.64898/2026.02.27.708412). eprint: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412.full.pdf>. URL: <https://www.biorxiv.org/content/early/2026/02/28/2026.02.27.708412>.

- [27] Aman Madaan et al. “Self-refine: Iterative refinement with self-feedback”. In: *Advances in neural information processing systems* 36 (2023), pp. 46534–46594.
- [28] Jörg Menche et al. “Uncovering disease-disease relationships through the incomplete interactome”. In: *Science* 347.6224 (2015), p. 1257601.
- [29] Meta AI. *The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation*. Meta AI Blog. 2025. URL: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/> (visited on 09/01/2026).
- [30] Peter J Mucha et al. “Community structure in time-dependent, multiscale, and multiplex networks”. In: *science* 328.5980 (2010), pp. 876–878.
- [31] Christopher J Mungall et al. “Uberon, an integrative multi-species anatomy ontology”. In: *Genome Biology* 13.1 (2012), R5.
- [32] Reiichiro Nakano et al. “Webgpt: Browser-assisted question-answering with human feedback”. In: *arXiv preprint arXiv:2112.09332* (2021).
- [33] NVIDIA. *NVIDIA Nemotron 3: Efficient and Open Intelligence*. White Paper. 2025. URL: <https://arxiv.org/abs/2512.20856>.
- [34] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: [2508.10925](https://arxiv.org/abs/2508.10925) [cs.CL]. URL: <https://arxiv.org/abs/2508.10925>.
- [35] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.
- [36] Pranav Putta et al. “Agent q: Advanced reasoning and learning for autonomous ai agents”. In: *arXiv preprint arXiv:2408.07199* (2024).
- [37] Yujia Qin et al. “Toollm: Facilitating large language models to master 16000+ real-world apis”. In: *arXiv preprint arXiv:2307.16789* (2023).
- [38] Rafael Rafailov et al. “Direct preference optimization: Your language model is secretly a reward model”. In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.
- [39] Jeff Rasley et al. “Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters”. In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 3505–3506.
- [40] G. van Rossum. *Python tutorial*. Tech. rep. CS-R9526. Amsterdam: Centrum voor Wiskunde en Informatica (CWI), May 1995.
- [41] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in neural information processing systems* 36 (2023), pp. 68539–68551.
- [42] John Schulman et al. “Proximal policy optimization algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).

- [43] Zhihong Shao et al. “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”. In: *arXiv preprint arXiv:2402.03300* (2024).
- [44] Noah Shinn et al. “Reflexion: Language agents with verbal reinforcement learning”. In: *Advances in neural information processing systems* 36 (2023), pp. 8634–8652.
- [45] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: [2601.03267](https://arxiv.org/abs/2601.03267) [cs.CL]. URL: <https://arxiv.org/abs/2601.03267>.
- [46] Kyle A. Sullivan et al. “MENTOR: Multiplex Embedding of Networks for Team-Based Omics Research”. In: *bioarXiv* (2024). DOI: [10.1101/2024.07.17.603821](https://doi.org/10.1101/2024.07.17.603821).
- [47] Shawn Zheng Kai Tan et al. “Brain Data Standards – A method for building data-driven cell-type ontologies”. In: *Scientific Data* 10 (2023), p. 50.
- [48] Kimi Team et al. *Kimi K2: Open Agentic Intelligence*. 2026. arXiv: [2507.20534](https://arxiv.org/abs/2507.20534) [cs.LG]. URL: <https://arxiv.org/abs/2507.20534>.
- [49] A. H. Vlot et al. “The MENTOR Interpretation Agent: From Network Embeddings to Mechanistic Narratives via Retrieval-Augmented LLMs”. In: *PASC 2025*. 2025.
- [50] Zhizheng Wang et al. “GeneAgent: self-verification language agent for gene-set analysis using domain databases”. In: *Nature Methods* 22.8 (2025), pp. 1677–1685.
- [51] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.
- [52] Leandro von Werra et al. *TRL: Transformers Reinforcement Learning*. <https://github.com/huggingface/trl>. 2020.
- [53] Shunyu Yao et al. “React: Synergizing reasoning and acting in language models”. In: *The eleventh international conference on learning representations*. 2022.
- [54] Andy B. Yoo, Morris A. Jette, and Mark Grondona. “SLURM: Simple Linux Utility for Resource Management”. In: *Job Scheduling Strategies for Parallel Processing*. Springer Berlin Heidelberg, 2003, pp. 44–60. DOI: [10.1007/10968987_3](https://doi.org/10.1007/10968987_3).
- [55] Jiayi Zhang et al. “Verbalized Sampling: How to Mitigate Mode Collapse and Unlock LLM Diversity”. In: *arXiv preprint arXiv:2510.01171* (2025).